JAVA SDE-2 INTERVIEW PREPARATION SERIES

VOLUME 18 OF 18 - PART J OF J

Advanced Java J: Distributed Systems and System Design

Capacity, Partitioning, Consistency, Kafka, Resilience, Sagas, Observability, and SLOs

FOUNDING AUTHOR

Vinay Reddy Kalluri

EDITOR-IN-CHIEF | CHIEF AUDITOR

Turn requirements into capacity estimates, data distribution, delivery semantics, resilience policies, observability, and defensible system-design trade-offs.

CAPACITY | SHARDING | CONSISTENCY | KAFKA | RESILIENCE | SLOS

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-25

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `18I – Advanced Java I – Persistence`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Part J completes the backend specialist track with a constraint-first system-design method spanning distributed data, messaging, failure control, and operations.

Readiness map

Decision	Evidence
START HERE	Scale or failure domains require partitioning and replication; Message ordering, replay, or delivery semantics affect correctness; Retries, rate limits, circuit breakers, or sagas cross services; A design needs measurable SLIs, SLOs, logs, metrics, and traces.
PREPARE FIRST	Parts 18H and 18I or equivalent backend experience; Concurrency, networking, and persistence fundamentals.
MOVE ON WHEN	Estimate capacity and choose partitioning, replication, and consistency contracts; Explain Kafka partitions, groups, offsets, rebalances, and delivery semantics; Design deadlines, retries, bulkheads, rate limits, backpressure, and sagas; Define observability signals, SLOs, and an incident evidence loop.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Volume Position

18I - Advanced Java I - Persistence	**YOU ARE HERE** **18J - Advanced Java J - System Design**	SERIES COMPLETE

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Capacity, Partitioning, Replication, and Consistency

Learning objectives

After this chapter, you should be able to:

- estimate throughput, storage, bandwidth, and concurrency with units and headroom;
- explain latency distributions and why queueing dominates near saturation;
- choose partition keys from routing, load distribution, locality, growth, and rebalancing needs;
- distinguish replication durability, availability, read scaling, and consistency goals;
- reason about quorums using explicit assumptions rather than the slogan `R + W > N`;
- apply CAP only to behavior during a network partition and PACELC to the latency/consistency tradeoff outside partitions;
- specify linearizable, sequential, causal, session, monotonic-read, read-your-writes, and eventual consistency contracts; and
- identify hot partitions, replica lag, split brain, failover, and repair requirements.

1. Numbers before boxes

Throughput and concurrency

If a service receives 50 million requests/day:

TEXT

```
average RPS = 50,000,000 / 86,400 ≈ 579
peak factor 8 -> design peak ≈ 4,630 RPS
```

If 30% of requests call the database for an average 40 ms, average concurrent database work near peak is:

TEXT

```
L = λW = (4,630 * 0.30 requests/s) * 0.040 s ≈ 56 operations
```

Little's Law is a steady-state relationship, not a pool-sizing command. Tail latency, bursts, retries, long transactions, multiple queries per request, and database useful concurrency require headroom and measurement. Keep units on every estimate.

Storage and bandwidth

For 10 million new events/day at 1.5 KiB average encoded size:

```text
raw/day ≈ 15 GiB
365 days ≈ 5.35 TiB
replication factor 3 ≈ 16 TiB before indexes, metadata, compaction, and headroom
```

Network egress for 4,630 RPS at 4 KiB response average is about 18 MiB/s before protocol overhead and replicas. Compression saves bandwidth at CPU/latency cost. Retention, secondary indexes, backups, multi-region copies, and write amplification can dominate raw payload.

Latency is a distribution

End-to-end latency includes queueing, application work, serialization, network, downstream queueing, storage, and retries. Percentiles cannot generally be added to obtain an end-to-end percentile; distributions and correlation matter. An average of 50 ms can coexist with p99 of seconds.

As utilization approaches a bottleneck's capacity, queueing rises nonlinearly. Therefore:

- bound queues and concurrency;
- keep work inside a deadline;
- measure p50/p95/p99 and saturation together;
- shed low-priority work before all requests miss their deadlines;
- avoid retry storms that increase arrival rate during failure.

2. Partitioning

Formal model

A partition function maps key `k` to partition `p`:

```text
p = f(k, partitionMetadata)
```

Hash partitioning spreads uniformly distributed keys and supports point routing. Range partitioning preserves ordered/range locality but can create hotspots for monotonic keys. Directory-based routing stores an explicit mapping and supports controlled placement at metadata complexity. Consistent-hashing families reduce remapping when membership changes but do not automatically solve skew, replicas, or operational rebalancing.

Partition-key decision rule

Evaluate:

1. routing: can the common request name the key?
2. distribution: does real traffic-not only key count-spread?
3. locality: which data and operations must be co-located?
4. growth: can one tenant/key exceed a partition?
5. ordering/transactions: what scope needs them?
6. rebalancing: how are data and requests moved safely?
7. secondary access: what fan-out or index is required?
8. privacy/residency: must certain data stay in a region?

Hot-partition walkthrough

Partitioning an event store by `customerId` appears uniform by number of customers. One enterprise tenant emits 30% of events, so its partition saturates. Adding partitions does not split that key. Options:

- use a compound shard key `(customerId, bucket)` and merge reads;
- dedicate capacity/partition placement for the whale tenant;
- separate tenant tiers/workloads;
- split time windows if query semantics permit;
- rate/admission limit the source;
- cache or aggregate hot reads.

Every split weakens single-key ordering/transaction locality and adds fan-out. State that tradeoff.

Rebalancing protocol

Moving partition ownership needs more than copying files:

1. choose source and target under capacity constraints;
2. copy a snapshot while tracking subsequent changes;
3. catch up log/delta;
4. update routing metadata with a version/epoch;
5. reject or redirect stale routers;
6. cut over reads/writes under the store's protocol;
7. verify checksums/counts and lag;
8. retire old replica after rollback window.

Epoch/fencing tokens prevent an old owner from accepting writes after a new owner takes control. Exact protocols belong to the chosen datastore.

3. Replication

Goals and costs

Replication can improve:

- availability under node failure;
- durability through independent copies;
- read throughput/locality;
- disaster recovery.

It also introduces lag, conflict, coordination, bandwidth/storage cost, failover state, and repair. Three copies in one failure domain do not provide regional disaster tolerance.

Common leadership models:

- **single leader:** one ordered write authority per shard; followers replicate; simple conflict model but leader/failover bottlenecks;
- **multi-leader:** writes accepted in several sites; lower local write latency but conflicts/loops/identity must be resolved;
- **leaderless/dynamo-style:** writes/reads contact subsets; application/store resolves versions and repairs; quorum slogans require careful assumptions;
- **consensus-replicated state machine:** a quorum agrees on an ordered log; strong semantics at coordination and availability/latency cost.

Do not infer a database's exact consistency from category alone. Read its documented protocol and configuration.

Failover and fencing

Failure detection is suspicion based on time and observation. A slow or partitioned leader may still be alive. Promoting a new leader without preventing the old from writing creates split brain. Use consensus lease/term/epoch and fencing at the storage boundary. A distributed lock token without fencing cannot stop a paused old owner from resuming and overwriting new work.

Failover objectives include:

- **RPO:** acceptable committed data loss;
- **RTO:** acceptable service restoration time;
- read/write availability during election;
- client retry and duplicate behavior;
- DNS/routing/cache convergence;
- validation before old primary rejoins;
- backup restore independent from replica corruption.

Replication is not backup: deletion or corruption can replicate.

4. Quorums without slogans

In a simplified replicated register with `N` replicas, write acknowledgement count `W`, and read response count `R`, the intersection condition `R + W > N` suggests a read and acknowledged write set overlap. But "overlap" yields the latest value only if:

- versions are comparable under a correct protocol;
- reads choose/repair the newest version;
- membership is stable or uses epochs;
- failed/partitioned writes follow defined rules;
- sloppy quorums/hinted handoff do not substitute unrelated nodes without accounting;
- clocks are not misused for ordering;
- concurrent writes/conflicts have resolution semantics;
- the system actually implements the assumed model.

Likewise, `W > N/2` causes write quorums to intersect in the simplified model, but does not by itself implement linearizability. Consensus protocols require terms, log matching, commit rules, leader fencing, and membership-change safety.

Example

`N=3, W=2, R=2` gives intersection. Client A's write reaches replicas 1 and 2. A read reaches 2 and 3; it sees the new version from 2 if the version is committed and selection is correct. During concurrent writes with last-write-wins timestamps, clock skew can discard an intended later business write. A logical version/vector/conflict merge may be needed depending on the datastore.

5. CAP and PACELC

CAP's precise question

During a network partition that prevents required nodes from communicating, a replicated system cannot both:

- provide linearizable behavior for all operations; and
- make every request to every partitioned side complete successfully.

The design chooses behavior per operation/configuration: reject or delay some operations to preserve a consistency contract, or accept operations with weaker/conflict semantics. "CA system" in a distributed network is often an unhelpful label because partitions are a fault to handle, not an optional feature toggle.

CAP does not say a system is always consistent or always available. It does not describe latency in normal operation, transaction isolation, or durability.

PACELC

PACELC adds: if there is a Partition, choose Availability or Consistency; Else, under normal operation, choose Latency or Consistency. Synchronous cross-region coordination can reduce stale reads but adds network latency and sensitivity to a remote quorum. Asynchronous replication improves local latency/availability but exposes lag and conflict windows.

Use these frameworks to ask concrete questions:

- Which operation and consistency model?
- What happens in a regional link partition?
- Which side accepts writes?
- What error/timeout does a client see?
- How is conflict reconciled?
- What normal-case coordination latency is paid?

6. Consistency models as client-visible contracts

- **linearizability:** each operation appears to take effect atomically between invocation and response, respecting real-time order;
- **sequential consistency:** operations appear in one total order preserving each process's program order, not necessarily real-time order;
- **causal consistency:** causally related operations are observed in causal order; concurrent operations may differ in order;
- **eventual convergence:** absent new updates and with successful communication/repair, replicas converge under the system's conflict rules;
- **read-your-writes:** a session sees its completed writes;
- **monotonic reads:** a session does not later observe an older version than it already saw;
- **monotonic writes:** a session's writes are applied in its order;
- **bounded staleness:** reads lag by at most a documented time/version bound under specified conditions.

Session guarantees can be implemented with sticky routing, version tokens, leader reads, quorum reads, or waiting for replica catch-up. Each has failure and latency tradeoffs. A client that switches regions can lose read-your-writes unless it carries a version/session token or routes to a sufficiently caught-up replica.

Product decision examples

- user profile edit: read-your-writes is highly visible; route to leader or use a version token;
- social feed ranking: eventual/bounded staleness may be acceptable;
- payment balance/authorization: stronger serialized invariant and idempotent commands;
- analytics dashboard: snapshot/as-of semantics may be better than pretending "current" across sources;
- inventory display: bounded stale display may be okay, but reservation must use authoritative concurrency control.

TRY IT | 7. Interview questions and model checkpoints

Q1. How do you choose a shard key?

Model checkpoint: query routing, traffic skew, co-location/transaction/order scope, growth, rebalancing, fan-out, and residency. Hashing IDs is not sufficient if one ID is hot.

Q2. Does R + W > N guarantee strong consistency?

Model checkpoint: only an intersection in a simplified model. Version selection, committed state, membership, conflict, sloppy quorum, repair, and actual datastore protocol determine semantics.

Q3. Explain CAP without "pick two."

Model checkpoint: during a communication partition, cannot make all operations on all sides both complete and preserve linearizability. Choice is per operation/configuration; normal-time latency is outside basic CAP.

Q4. How do you provide read-your-writes from replicas?

Model checkpoint: leader read, sticky session, carry observed version and wait/route to caught-up replica, or stronger quorum/protocol. Bound the wait and handle failover.

SDE-2 follow-ups

1. A tenant grows beyond one shard. Design split and routing migration while preserving identity.
2. A region is isolated for 20 minutes. Specify writes, reads, client errors, and reconciliation for orders versus product catalog.
3. Estimate capacity when retries double write traffic and replication is three-way.
4. Distinguish linearizability, serializability, and strict serializability in an interview.

TRY IT | 8. Exercises

1. Estimate one year of storage, peak network, and concurrent DB calls for a workload; show units and three sensitivity scenarios.
2. Compare hash, range, time, and tenant partitioning for an event service. Identify hot keys and query fan-out.
3. Simulate N=5 quorum choices and list assumptions needed for a latest-value read.
4. Write client-visible contracts for regional failover under linearizable and eventual product choices.
5. Design a rebalancing runbook with epochs, validation, throttling, rollback, and observability.

RECAP | 9. Summary checklist

- [ ] Estimates show units, peaks, replication, indexes, retention, and headroom.
- [ ] Latency is treated as a distribution with queueing and saturation.
- [ ] Partition key handles traffic skew, locality, growth, and rebalance.
- [ ] Replication goals and failure domains are explicit.
- [ ] Failover includes fencing, client semantics, RPO/RTO, and repair.
- [ ] Quorum claims list protocol assumptions.
- [ ] CAP is scoped to partition behavior; PACELC covers normal latency tradeoff.
- [ ] Every consistency model is phrased as an observable client contract.

10. Multi-region decision laboratory

Assume users in North America and Europe create and read orders. Product asks for low latency and "no lost orders." Do not jump directly to active-active writes. Decompose the promises:

- accepted create must be durable under which failures?
- can one order receive commands from both regions concurrently?
- must a European client immediately read a North American write?
- can the service reject writes during inter-region partition?
- what RPO/RTO applies to region loss?
- where may personal data reside?

Option A: home-region single writer

Assign each order/customer a home region. Route writes there; asynchronously replicate read views elsewhere. Carry home/observed version in session/routing metadata.

Benefits: one write authority simplifies order invariants and per-order sequence. Costs: remote write latency for traveling users, home-region outage/failover, routing metadata, stale remote reads. During link partition, non-home region can serve bounded-stale reads but rejects/queues writes according to policy. Promotion uses a term/epoch and fencing so the old home cannot resume writes.

Option B: synchronous cross-region quorum

Commit requires replicas across failure domains. This can reduce RPO and provide stronger global semantics under its consensus protocol. It adds wide-area round-trip latency to writes and can reject them if quorum is unavailable. "Three replicas" is insufficient detail: placement and quorum membership determine which region failures are tolerated.

Option C: multi-writer with conflict resolution

Both regions accept writes. This improves local write availability/latency during partition but requires conflict semantics. Last-write-wins based on wall clock is dangerous for order transitions. A domain merge might be possible for independent profile fields, but `CANCELLED` versus `SHIPPED` is not a mechanical merge. Escrow/reservation, single-aggregate leadership, CRDTs for suitable data, or workflow reconciliation may narrow the conflict domain.

Partition walkthrough

At `t0`, link fails. Europe accepts a cancellation while North America ships the order.

- Under home-region consistency, Europe should route/fail the write if North America is home; it must not acknowledge cancellation it cannot serialize.
- Under synchronous quorum, one or both sides lack quorum and rejects/blocks within deadline.
- Under multi-writer availability, both can commit and reconciliation needs a business outcome: stop shipment if possible, return/refund, audit, and prevent stale state overwrite.

CAP appears as a concrete product choice, not a label.

Disaster recovery is separate

Replicas can faithfully copy accidental deletion, bad migration, or application corruption. Maintain tested backups/point-in-time recovery, immutable/offline policy as required, restore drills, schema/config artifacts, secret/key recovery, and reconciled restart. Record RPO/RTO from observed drills, not architecture diagrams.

Multi-region observability

Track per region and cross-region:

- replication apply/commit lag in time and versions;
- quorum/leader/term changes and fencing failures;
- remote routing latency and error;
- stale-read/session-token waits;
- conflict/reconciliation count and oldest age;
- data-residency policy violations;
- regional SLI/error-budget burn;
- failover RTO and post-failover validation.

Do not tag every tenant in metrics. Use tenant tier/region and investigate identities through controlled logs/traces.

Laboratory checkpoint

A defensible multi-region answer specifies one operation's write authority, acknowledgement rule, read source, partition response, conflict behavior, failover fencing, RPO/RTO, and data residency. "Active-active" is a topology adjective, not a consistency contract.

Primary references

- Gilbert and Lynch, "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services": https://dl.acm.org/doi/10.1145/564585.564601
- Ongaro and Ousterhout, "In Search of an Understandable Consensus Algorithm (Raft)": https://raft.github.io/raft.pdf
- Apache Kafka Design Documentation, replication background: https://kafka.apache.org/documentation/#design_replicatedlog
- OpenJDK Java 21 documentation: https://docs.oracle.com/en/java/javase/21/

> **Version boundary:** partitioning, quorum, leader election, consistency, and failover are datastore-specific contracts. Categories in this chapter are reasoning tools, not guarantees about a named product. Java examples in this mini-book compile on Java 21; later JDK features are separated as optional deltas.

SDE-2 CORE CHAPTER

Chapter 2 - Kafka Partitions, Consumer Groups, and Delivery Semantics

Learning objectives

After this chapter, you should be able to:

- model a Kafka topic as ordered partition logs with replicated durability configured at broker/topic/producer layers;
- select a message key from ordering, parallelism, skew, and evolution needs;
- explain consumer groups, assignments, polling, offset position, committed offsets, and rebalances;
- place offset commits relative to effects to obtain an explicit at-most-once or at-least-once failure window;
- explain what producer idempotence and Kafka transactions do and do not guarantee;
- evolve event schemas under producer/consumer compatibility rules;
- design retry, quarantine/DLQ, replay, and idempotent handling as operational protocols; and
- use Spring for Apache Kafka without hiding broker semantics behind annotations.

1. The log and partition model

A topic has one or more partitions. Each partition is an append-only ordered sequence addressed by offsets:

```text
partition 0: [offset 0][1][2][3]...
partition 1: [offset 0][1][2]...
```

Order is defined *within a partition*, not globally across a topic. An offset identifies a position, not a message identity and not a timestamp. Retention can remove old records even though offsets remain conceptual positions. Compaction, when configured, retains records according to key/tombstone and broker policy; it is not a full audit log guarantee.

Partitions provide parallelism and the unit of replicated leadership/assignment. More partitions can increase parallel consumption but also add metadata, files, replication/recovery work, rebalance scope, and ordering fragmentation. Partition count changes can alter key-to-partition mapping under common partitioners, so a key ordering/history contract needs a migration plan.

Producer send model

A producer serializes a key and value, selects a partition, batches/compresses records, sends to the partition leader, and receives an acknowledgement under configured durability semantics. Correctness depends on the combination of:

- topic replication factor and in-sync replica policy;
- producer acknowledgement configuration;
- retry and delivery timeout;
- idempotence/transactions;
- broker failure and leader election policy;
- error handling in application code.

Never say "Kafka write succeeded" without defining which acknowledgement completed and what durability/failure domain that implies for the deployed cluster.

2. Keys, ordering, and hot partitions

Key decision rule

Choose the smallest business identity whose events require relative order. For order lifecycle events, `orderId` is a natural key:

```text
OrderPlaced(O-7) -> PaymentAuthorized(O-7) -> OrderConfirmed(O-7)
```

Different orders can process in parallel. Keying by `customerId` would serialize all of one customer's orders and can create hotspots. A null key may be spread by the producer but offers no stable per-entity routing contract.

Per-key order is preserved only along a compatible path:

- all relevant records select the same partition;
- producer retries/configuration preserve documented ordering behavior;
- consumers do not apply records concurrently out of order within the key;
- retries do not divert one failed record while later same-key records commit incorrectly;
- repartitioning or topic migration preserves the contract.

Hot-key walkthrough

A global tenant configuration event uses `tenantId`; one tenant generates half the traffic. Its partition is saturated while others are idle. Random salting spreads load but destroys simple per-tenant order. Alternatives:

- subdivide by a business subkey and make consumers order only within subkey;
- isolate the heavy tenant/topic;
- aggregate or rate-limit producer traffic;
- use more partitions only if there are enough independent keys;
- design sequence/version conflict handling so out-of-order across buckets is acceptable.

Partition skew is measured in bytes, records, processing time, and lag-not key count alone.

3. Consumer groups and offsets

Group model

Within one consumer group, each partition is assigned to at most one active group member at a time under the group protocol. A member can own multiple partitions; if members exceed partitions, some are idle. Separate groups consume independently.

The consumer has concepts that are often conflated:

- **current position:** next offset the consumer will return locally;
- **committed offset:** stored recovery position for the group;
- **high watermark/end position:** broker-side boundary used to define lag under the implementation's metrics;
- **processed effect:** application's external state, which Kafka does not infer from method return.

Committing an offset means "on recovery, the group may resume from here" under the API protocol. It does not prove a database transaction committed unless the application coordinated those actions through an explicit design.

Poll loop contract

Simplified native client sketch - dependency-requiring:

JAVA

```java
while (running) {
    ConsumerRecords<String, OrderEvent> batch = consumer.poll(pollTimeout);
    for (TopicPartition partition : batch.partitions()) {
        List<ConsumerRecord<String, OrderEvent>> records =
                batch.records(partition);
        for (ConsumerRecord<String, OrderEvent> record : records) {
            handler.applyIdempotently(record.key(), record.value());
        }
        long next = records.get(records.size() - 1).offset() + 1;
        consumer.commitSync(Map.of(
                partition, new OffsetAndMetadata(next)));
    }
}
```

This commits per-partition after processing. Real code must handle empty batches, wakeup/shutdown, retriable/fatal broker errors, partition revocation, partial failures, time budgets, and poison records. Long processing must remain within group liveness/max-poll protocol or use supported pause/worker/handoff design carefully.

Do not process a partition concurrently without a sequencing and commit-watermark algorithm. If offsets 10 and 11 run in parallel and 11 finishes first, committing 12 before 10's effect is durable can lose 10 on crash.

4. Rebalances

Group membership or subscription/metadata changes can reassign partitions. Depending on client/broker protocol and configuration, rebalancing may revoke broad assignments or transfer incrementally/cooperatively. Exact protocol support is version-sensitive.

An assignment lifecycle needs:

1. stop accepting new work for revoked partitions;
2. complete/cancel/drain in-flight work under a deadline;
3. durably record safe processed offsets if application-managed;
4. release partition-owned state/resources;
5. initialize state for newly assigned partitions;
6. resume polling without exceeding liveness constraints.

Rebalance failure walkthrough

Consumer A processes partition 2 offset 100 slowly. A rebalance revokes it and assigns partition 2 to B. If A continues and writes after B processes the same/newer key, stale effects can overwrite new state. Idempotency by event ID prevents duplicates but not necessarily stale ordering. Use source sequence/version conditions or fencing ownership when effects require it, and stop revoked work promptly.

Rebalance storms can come from slow polls, unstable instances, aggressive timeouts, deployments, partition changes, or coordinator/broker issues. Observe rebalance count/duration, assignment churn, processing time, poll interval, and lag rather than blindly increasing timeouts.

5. Delivery semantics and failure windows

At-most-once

Commit/advance recovery position before applying the effect:

```text
receive -> commit offset -> effect
```

Crash after commit but before effect loses the message. Suitable only when occasional loss is acceptable or another replay source exists.

At-least-once

Apply effect, then commit:

```text
receive -> effect -> commit offset
```

Crash after effect before commit repeats the record. This is common and requires an idempotent effect or deduplication. For a database consumer, insert an inbox/event ID and domain transition in one local transaction:

```sql
begin;
insert into consumer_inbox(consumer_name, event_id) values (?, ?)
    on conflict do nothing;
-- Only if inserted: apply the state transition with source version.
commit;
```

Then commit Kafka offset. Duplicate delivery repeats the DB transaction, sees inbox, and becomes a no-op. If DB commits and offset commit fails, replay is safe.

Producer idempotence

Kafka's idempotent producer protocol prevents duplicate writes caused by supported producer retries within its producer-session/partition semantics and preserves relevant ordering under documented configuration. It does not deduplicate two independent business calls that produce the same logical event. Use a stable event ID/outbox for business idempotency.

Kafka transactions/exactly-once

Kafka transactions can atomically write records to Kafka partitions and commit consumed offsets for consume-process-produce pipelines when producers/consumers use the required transaction and isolation settings. The practical guarantee is scoped to Kafka resources and the configured clients. It does not atomically include an arbitrary relational database or HTTP call.

State precisely:

- which inputs/outputs are Kafka records;
- whether consumers use committed-only isolation;
- transactional identity/epoch behavior across restarts;
- what external side effects exist;
- how duplicates/retries at the business/API layer are handled.

"Exactly once" is not a magic absence of retries. It is a protocol that makes selected writes/offsets visible atomically and filters aborted work for participating consumers.

6. Event schema evolution

Event contract

An event is an immutable statement about something that happened, with:

```text
eventId, eventType, schemaVersion, aggregateId/key,
occurredAt, producer, trace/correlation metadata, payload
```

Avoid copying request DTOs or database entities onto the wire. Define field meaning, units, timezone, optionality, enum/unknown handling, identity, and privacy. `occurredAt` is domain/event time; broker append time and processing time are different.

Compatibility depends on serializer/schema system, but general rules include:

- add optional/defaultable fields when old consumers can ignore them;
- do not reuse a field name/number with a new meaning;
- do not silently change units or enum semantics;
- preserve readers during rolling producer/consumer deployment;
- use new event type/version or dual publishing when meaning is incompatible;
- retain raw input/replay ability only under privacy and retention policy;
- test representative old/new readers and writers.

A schema registry can enforce syntactic compatibility rules, but cannot prove semantic compatibility. Renaming `amountCents` to `amount` while changing units can pass some schemas and still break money.

7. Retry topics, DLQ, and replay

Classify failures

- transient dependency failure: retry with bounded exponential backoff/jitter;
- rate/capacity failure: pause/backpressure; retries alone amplify load;
- validation/schema poison record: quarantine with reason and payload reference under security policy;
- authorization/business rejection: terminal domain outcome, not infrastructure retry;
- code defect: halt/contain and deploy fix; repeated retry burns capacity.

Blocking a partition preserves same-partition order but one poison record can stop unrelated keys. Moving to retry topics improves throughput but later same-key events can overtake. Options:

- keep ordered retries for workflows that require it;
- version-check state so stale events are rejected;
- partition retry stream by same key and coordinate delay;
- separate independent workloads/keys;
- quarantine with an explicit manual repair that accounts for later events.

DLQ is a workflow

A useful quarantine record includes original topic/partition/offset, key, event ID/schema, failure code, first/last failure times, attempt count, handler version, and a secure reference or redacted payload. Operations require:

- alert/ownership and age/count SLO;
- investigation tooling with authorization/audit;
- fix-forward, discard, or replay decision;
- replay idempotency and rate control;
- prevention tracking;
- retention/privacy deletion.

Blindly replaying a DLQ into the original topic can recreate the incident and violate order. Use a controlled replay producer and observe effects.

8. Spring Kafka boundary - dependency-requiring

JAVA

```java
@KafkaListener(topics = "order-events", groupId = "fulfillment-v3")
void onOrderEvent(ConsumerRecord<String, OrderEvent> record,
                  Acknowledgment acknowledgment) {
    handler.applyInDatabaseTransaction(
            record.key(), record.value(), record.topic(),
            record.partition(), record.offset());
    acknowledgment.acknowledge();
}
```

The annotation does not define delivery semantics by itself. Container acknowledgement mode, listener transaction integration, error handler/retry policy, concurrency, assignment, deserializer behavior, and broker/client settings determine execution. Acknowledge only after the durable effect according to the chosen protocol. Integration-test crash windows with the actual framework/client versions.

TRY IT | 9. Interview questions and model checkpoints

Q1. How many consumers can one group use effectively?

Model checkpoint: up to the number of partitions for direct partition assignment at a time; extra consumers are idle. Processing concurrency can be designed inside a member but must preserve per-partition/key order and safe offset watermark.

Q2. Why might a record be processed twice?

Model checkpoint: effect succeeded and offset commit failed/crash occurred, producer/business duplicated, rebalance repeated uncommitted work, or retry policy redelivered. Use event identity/source version and atomic inbox+effect.

Q3. Does idempotent producer make a create-order API idempotent?

Model checkpoint: no. It addresses producer retry duplicates within Kafka protocol scope. API request identity/outbox/business event ID addresses repeated business commands.

Q4. What is the ordering guarantee?

Model checkpoint: records in a partition have broker log order; same-key producer partitioning commonly co-locates records. End-to-end effects need serial handling/versioning and retry/rebalance design. No topic-global order.

SDE-2 follow-ups

1. Process one partition with a worker pool while committing only the highest contiguous completed offset.
2. Increase partition count without breaking per-customer workflow semantics.
3. Combine a relational outbox with Kafka producer idempotence and consumer inbox; enumerate all duplicate windows.
4. Design a schema-breaking event migration with old consumers running for two weeks.

TRY IT | 10. Exercises

1. Draw crash points for commit-before, commit-after, and Kafka transactional consume-transform-produce.
2. Design a per-key ordering policy across main, retry, and quarantine topics.
3. Implement a database inbox schema with retention, tenant scope, and source-version checks.
4. Write a rebalance callback plan that drains work without exceeding the poll/liveness protocol.
5. Define dashboards for partition lag, processing latency, rebalance, producer errors, under-replication, and DLQ age.

RECAP | 11. Summary checklist

- [ ] Partition count and key balance order, skew, and parallelism.
- [ ] Producer durability is specified through replication and acknowledgement settings.
- [ ] Offset position, commit, and durable business effect are not conflated.
- [ ] Rebalances stop/fence revoked work and preserve safe recovery positions.
- [ ] At-most/at-least/exactly-once claims enumerate failure windows and scope.
- [ ] External effects are idempotent or deduplicated in a local atomic boundary.
- [ ] Schema compatibility includes semantics, not only registry validation.
- [ ] Retry and DLQ preserve or deliberately relax ordering with operations ownership.

Primary references

- Apache Kafka Documentation: https://kafka.apache.org/documentation/
- Apache Kafka Producer configuration: https://kafka.apache.org/documentation/#producerconfigs
- Apache Kafka Consumer configuration: https://kafka.apache.org/documentation/#consumerconfigs
- Apache Kafka Design: https://kafka.apache.org/documentation/#design
- Spring for Apache Kafka Reference: https://docs.spring.io/spring-kafka/reference/

Version boundary: consumer group protocols, assignment strategies, broker defaults, idempotence defaults, transaction behavior, partitioner behavior, Spring listener APIs, and retry facilities evolve. This chapter states conceptual contracts; verify exact broker/client/framework versions from their official documentation. Application Java baseline is 21.

SDE-2 CORE CHAPTER

Chapter 3 - Deadlines, Retries, Circuit Breakers, Backpressure, and Sagas

Learning objectives

After this chapter, you should be able to:

- propagate one end-to-end deadline through queues, attempts, and downstream calls;
- classify retryable failures and calculate retry amplification;
- distinguish circuit breakers, bulkheads, rate limits, concurrency limits, and timeouts;
- implement token-bucket reasoning and choose a distributed enforcement boundary;
- design backpressure from bounded capacity rather than unbounded queues;
- specify a saga as durable forward and compensating transitions with idempotency; and
- walk through partial failure without confusing timeout, cancellation, rollback, and non-occurrence.

1. Deadline is a correctness boundary

A timeout is a local waiting limit. A deadline is an absolute latest useful completion time. If a request arrives with deadline `T_d`, each component computes:

```text
remaining = T_d - monotonicNow
downstream budget <= remaining - localResponseMargin
```

Do not propagate a wall-clock instant between machines without accounting for clock behavior/protocol. Many RPC frameworks carry a relative/absolute deadline under documented semantics. Inside Java, use monotonic elapsed-time sources such as `System.nanoTime()` for durations rather than subtracting wall-clock timestamps.

Budget example

An API has 800 ms server budget:

```text
admission/queue:       50 ms
local validation:     20 ms
inventory call:      250 ms max
payment call:        300 ms max
DB commit/outbox:    100 ms
serialization/margin:80 ms
```

These are caps, not a reason to wait each maximum sequentially. Before each call, use remaining budget. If queueing consumed 400 ms, starting a 300 ms payment plus commit may be pointless. Reject/abort before performing a side effect that cannot return usefully.

Cancellation is cooperative and races with completion. A client timeout does not prove downstream work stopped. Every side-effecting command needs identity/status reconciliation. Interrupting a Java thread does not roll back remote operations; code must preserve interruption policy and close/abort only through supported APIs.

2. Retries

Retry decision rule

Retry only when all are true:

1. failure is plausibly transient and classified from a stable signal;
2. another attempt can succeed within the remaining deadline;
3. operation is idempotent or protected by an idempotency key/deduplication;
4. retry load will not violate a budget or worsen overload;
5. attempts and elapsed time are bounded;
6. metrics expose attempts and terminal outcome.

Good candidates: connection reset before a request is accepted, selected leader-change/unavailable signals, classified serialization/deadlock victim for a local transaction. Bad candidates: validation, permission denial, invariant conflict needing user input, deterministic serialization bug, arbitrary `500` for a non-idempotent command.

Exponential backoff with jitter

One full-jitter form:

```text
cap_i = min(maxBackoff, base * 2^i)
sleep_i = random(0, cap_i)
```

Jitter prevents synchronized clients from retrying in lockstep. Respect server retry guidance if safe, but bound it by the caller deadline. Avoid overflow when calculating powers.

Amplification

If each layer performs up to three attempts across gateway, service, and client, one user action can cause `3 * 3 * 3 = 27` downstream attempts. Centralize retry ownership where possible. Define a retry budget as a fraction/count of normal traffic, not only attempts per request.

Failure walkthrough

Payment provider receives command and charges, but response is lost. A blind retry with a new identity charges again. Correct protocol sends stable merchant operation ID; provider returns/reconciles the previous outcome. The order workflow stores "payment unknown/pending reconciliation" rather than assuming timeout means failure. Compensation/refund is a separate idempotent business action, not network rollback.

3. Circuit breakers

A circuit breaker is a state machine protecting callers/dependency from repeated work that is unlikely to succeed:

```text
CLOSED --failure threshold/window--> OPEN
OPEN --cooldown--> HALF_OPEN
HALF_OPEN --limited successes--> CLOSED
HALF_OPEN --failure--> OPEN
```

The contract needs:

- which outcomes count as failures (not client validation);
- sliding window/minimum sample size;
- failure/slow-call threshold;
- open duration and half-open probe concurrency;
- fallback/rejection response;
- metrics and manual/automatic recovery;
- scope: per instance, dependency, endpoint, tenant, or region.

Per-instance breakers can disagree; that may be acceptable because each protects local resources. A globally synchronized breaker can become a control-plane dependency. Do not use a breaker for a healthy-but-saturated service when admission/rate control is the needed signal.

Fallback must be semantically safe. Returning stale catalog data may be acceptable. Returning a guessed authorization decision or stale bank balance is not. Silent empty lists turn outages into data loss signals.

4. Bulkheads and capacity isolation

A bulkhead partitions scarce capacity:

- separate connection pools for critical and batch paths;
- separate executors/concurrency semaphores by dependency;
- tenant quotas;
- workload queues with weighted admission;
- dedicated Kafka consumer groups/resources.

Without isolation, a slow recommendation provider can consume every request thread/connection and take down order status. A semaphore limit around the dependency bounds in-flight calls; callers that cannot acquire within the deadline fail fast or degrade.

Bulkheads reduce maximum utilization and require capacity allocation. Too many tiny pools strand resources. Review with measured demand and priority policy.

5. Rate and concurrency limiting

Token bucket

A token bucket has capacity `B` and refill rate `r` tokens/second. At elapsed time `Δt`:

```text
tokens = min(B, tokens + r * Δt)
admit cost c iff tokens >= c; then tokens -= c
```

It permits bursts up to capacity while limiting long-term average. Use monotonic elapsed time. Decide rounding/fractional tokens, maximum idle accumulation, and weighted request cost.

Other policies:

- fixed window: simple but boundary bursts;
- sliding window log/counter: smoother, more state/cost;
- leaky bucket: controls departure rate/queueing;
- concurrency limit: caps in-flight work, often better when latency varies;
- adaptive concurrency: adjusts from measured latency/queueing under careful stability controls.

Distributed limit placement

- edge/gateway: protects downstream early; needs authenticated identity and failure policy;
- per-instance: cheap but total scales with instances and load balance;
- centralized Redis/service: coordinated limit but adds latency/dependency/hot keys;
- hierarchical: local leases/budgets allocated from global policy, trading precision for scale.

Define behavior when limiter storage is unavailable: fail open risks overload/abuse; fail closed risks outage. Choose by endpoint (login/payment versus low-risk read), use emergency local limits, and observe.

Return `429` with useful retry semantics only when the caller can retry safely. Internal callers need backpressure signals, not a synchronized tight retry loop.

6. Backpressure and bounded queues

Backpressure prevents producers from creating more in-flight work than consumers can finish. It can be pull-based demand, bounded blocking, rejection, pause/resume, load shedding, or rate feedback.

An unbounded queue preserves acceptance by spending memory and deadline. At arrival `λ > service μ`, backlog grows at `λ − μ`. Once queued wait exceeds the request deadline, doing work wastes capacity and increases recovery time.

Design each queue with:

- capacity in items and bytes;
- priority/fairness and tenant quotas;
- maximum queue age/deadline expiration;
- rejection/drop policy;
- producer feedback;
- retry ownership;
- drain/shutdown behavior;
- depth, age, throughput, rejection, and saturation metrics.

Kafka lag is a durable backlog, but retention is finite and downstream recovery throughput must exceed arrivals. Simply adding consumers helps only until partition count or downstream capacity binds. Pause partitions when a dependency is saturated, continue polling under supported patterns, and avoid exceeding consumer liveness rules.

7. Sagas

Model

A saga is a sequence of local transactions connected by durable commands/events. If a later step fails, compensating actions attempt to semantically reverse or offset earlier committed effects:

```text
T1 ReserveInventory -> T2 AuthorizePayment -> T3 ConfirmOrder
        C1 ReleaseInventory <- C2 Void/RefundPayment
```

Compensation is not rollback:

- inventory may have been unavailable to others temporarily;
- an authorization may require void, a captured payment refund;
- refund can fail and require reconciliation;
- notifications cannot be "unsent";
- policy may choose manual intervention rather than compensation.

Every transition and compensation needs a stable command/event ID, idempotent handling, allowed-source-state predicate, timeout/retry, and audit. Store saga state durably with version/sequence.

Orchestration versus choreography

Orchestration: a coordinator sends commands and records transitions. Pros: visible workflow, timers, centralized policy. Cons: coordinator coupling/availability and possible "god service."

Choreography: services react to events. Pros: decoupled local evolution. Cons: implicit global flow, cycles, difficult status/timeouts, event sprawl. Complex business workflows usually benefit from an explicit durable process manager even if communication is event-based.

Order saga walkthrough

States:

```text
PENDING -> INVENTORY_RESERVED -> PAYMENT_AUTHORIZED -> CONFIRMED
       \-> REJECTED
INVENTORY_RESERVED -> COMPENSATING -> RELEASED/FAILED_RECONCILIATION
PAYMENT_AUTHORIZED -> COMPENSATING -> VOIDED/REFUND_PENDING
```

1. Create order and outbox `ReserveInventory` command atomically.
2. Inventory consumes idempotently and emits reserved/rejected with source command ID.
3. Coordinator accepts only expected order version/state; duplicates no-op.
4. On reserved, append `AuthorizePayment` with stable operation ID.

5 Provider timeout yields `PAYMENT_UNKNOWN`, not immediate failure; query/reconcile using operation ID.

6 On terminal payment failure, command inventory release.

7 Repeated release is idempotent; reservation has an expiry as a safety net.

8 Operators see sagas stuck beyond age SLO and can safely replay/repair.

Concurrent cancel request is another transition. Use optimistic version and define whether cancellation before/after capture triggers release/refund.

8. Java/Spring implementation boundaries

Dependency-requiring resilience libraries can implement mechanics, but configuration needs domain semantics:

JAVA

```java
@Service
final class PricingGateway {
    private final HttpClient client;
    private final Bulkhead bulkhead;
    private final CircuitBreaker breaker;

    Price quote(Command command, Deadline deadline) {
        return breaker.executeSupplier(() ->
            bulkhead.executeSupplier(() ->
                client.quote(command, deadline.remaining())));
    }
}
```

Decorator order matters. If retry occurs inside a bulkhead permit, one request can hold capacity through backoff; if outside, each attempt reacquires. Metrics should distinguish logical request from attempt. Time limiter cancellation may not interrupt underlying blocking I/O. Test failure windows on the real client.

Java 21 virtual threads make blocking code cheaper in thread terms but do not remove deadlines, idempotency, connection limits, queues, or downstream saturation. Structured concurrency APIs have differed across later JDK releases; this book does not assume preview APIs.

TRY IT | 9. Interview questions and model checkpoints

Q1. Timeout versus deadline?

Model checkpoint: timeout limits one wait; deadline bounds the whole operation. Recompute remaining budget and leave commit/response margin. Timeout does not prove non-execution.

Q2. Circuit breaker versus rate limiter?

Model checkpoint: breaker rejects because recent evidence predicts dependency failure; limiter enforces allowed demand/capacity regardless of health. Bulkhead isolates resources; timeout bounds waiting.

Q3. How do you retry safely?

Model checkpoint: classify transient failure, stable operation identity/idempotency, bounded attempts/elapsed budget, exponential backoff+jitter, capacity-aware policy, metrics, and reconciliation for unknown outcome.

Q4. Is saga compensation guaranteed?

Model checkpoint: compensation is another fallible local transaction. Retry idempotently, persist progress, reconcile/manual repair, and model irreversible effects.

SDE-2 follow-ups

1. Compute worst-case downstream attempts for three retrying layers and redesign ownership.
2. Design rate limits for global, tenant, user, and expensive-route constraints without a hot central key.
3. A breaker opens due to client `400`s. Fix classification and describe test evidence.
4. Handle simultaneous payment callback, timeout reconciliation, and user cancellation in one saga state machine.

TRY IT | 10. Exercises

1. Allocate an end-to-end 1-second deadline across two parallel and one sequential dependency, including queue margin.
2. Implement a Java 21 monotonic token bucket and assert burst/refill behavior.
3. Design a bounded executor policy for critical and batch tasks; calculate memory and queue-age limits.
4. Draw an order saga with every command ID, local transaction, timeout, duplicate, compensation, and terminal/manual state.
5. Write a chaos matrix for slow dependency, reset, lost response, partial region, retry storm, and recovery.

RECAP | 11. Summary checklist

- [ ] One end-to-end deadline controls queueing, attempts, and commits.
- [ ] Retry classification, identity, attempts, elapsed time, backoff, and amplification are explicit.
- [ ] Breakers, bulkheads, limits, and timeouts have distinct purposes.
- [ ] Every queue is bounded by capacity, age, and rejection policy.
- [ ] Rate limiting has a scope and unavailable-store policy.
- [ ] Side-effect timeout produces unknown/reconciliation state where necessary.
- [ ] Saga states, commands, compensations, duplicates, and manual recovery are durable.
- [ ] Framework decorators are tested in their actual order and execution model.

Primary references

- RFC 9110, HTTP method/idempotency and status semantics: https://www.rfc-editor.org/rfc/rfc9110
- Reactive Streams specification (backpressure contract): https://www.reactive-streams.org/
- Java SE 21 API: https://docs.oracle.com/en/java/javase/21/docs/api/
- Spring Framework Reference, resilience-related integration boundaries: https://docs.spring.io/spring-framework/reference/

> **Version boundary:** circuit-breaker libraries, HTTP clients, Spring integrations, virtual-thread support, and structured-concurrency APIs vary. The patterns specify behavior independent of a library. All complete companion code targets final Java 21 APIs and avoids later-JDK preview features.

SDE-2 CORE CHAPTER

Chapter 4 - Structured Logs, Metrics, Traces, and SLO Engineering

Learning objectives

After this chapter, you should be able to:

- design telemetry from questions and failure modes rather than logging everything;
- produce structured, correlated, redacted logs with bounded event volume;
- choose metric types and low-cardinality dimensions;
- explain Micrometer as an instrumentation facade/observation API and OpenTelemetry as a telemetry model, API/SDK, context, and protocol ecosystem;
- use RED for request-driven services and USE for resources;
- define an SLI with a precise eligible-event population, good-event rule, window, and source;
- calculate error budget and multi-window burn-rate intuition; and
- lead an incident from user impact through hypotheses, evidence, mitigation, validation, and prevention.

1. Observability is question-answering capacity

Telemetry is useful when it can answer:

- Are users failing or slow?
- Which route, tenant tier, region, version, or dependency changed?
- Is the bottleneck demand, CPU, memory, queue, connection, lock, network, storage, or downstream?
- Which request/event followed the failing path?
- Did mitigation restore the SLI?
- Can we reproduce and prevent the condition?

Logs, metrics, and traces are complementary:

- **metrics:** bounded-dimensional aggregates; cheap trend/alert foundation;
- **logs:** discrete structured events and diagnostic context;
- **traces:** causally related spans for sampled/distributed executions;
- **profiles/dumps:** code/runtime resource evidence when CPU, allocation, locks, or threads require depth.

Instrument semantic boundaries-request, queue, use case, database, broker, remote call-not every method. Telemetry consumes CPU, allocation, network, storage, and human attention; it needs budgets and failure behavior.

2. Structured logging

Event contract

A structured log record can contain:

JSON

```
{
  "timestamp": "2026-01-01T12:00:00Z",
  "level": "WARN",
  "service": "orders",
  "environment": "prod",
  "event": "payment_outcome_unknown",
  "traceId": "...",
  "orderId": "O-91",
  "provider": "primary",
  "elapsedMs": 287,
  "outcome": "TIMEOUT",
  "attempt": 1
}
```

Use stable field names/types and an event code. Message prose can change; automation should use structured outcome fields. Include IDs only when policy permits; avoid tokens, credentials, card data, full request/response, personal notes, SQL parameter dumps, and secrets in exception messages.

Correlation and context

Trace context should propagate through supported HTTP/messaging mechanisms. A business operation ID/event ID can correlate retries across traces. Do not use trace ID as idempotency identity: trace sampling/lifetime semantics differ.

Thread-local logging context does not automatically cross executors/reactive pipelines/virtual-thread task boundaries correctly. Use framework-supported context propagation and clear scopes. Verify that asynchronous callbacks and Kafka records preserve only necessary context.

Logging mistakes

- logging the same exception at repository, service, controller, and gateway;
- dynamic message templates that defeat grouping;
- metric-like high-volume success logs instead of counters;
- logging every retry as error when terminal operation succeeds;
- unbounded stack traces for expected validation;
- using raw URL/query strings containing IDs as route labels;
- swallowing a failure after logging without returning/recording an outcome;
- synchronous remote log append on the critical path without backpressure policy.

Log once at the boundary that owns classification, with enough causal context. Sampling must preserve rare critical errors according to policy.

3. Metrics and cardinality

Metric types

- **counter:** monotonically increasing event count; derive rate over time;
- **gauge:** sampled current value such as queue depth; can jump and vanish with process;
- **histogram/distribution:** counts observations in buckets, enabling aggregate percentiles and SLO thresholds;
- **timer:** duration distribution plus count, represented by the instrumentation backend;
- **long-task timer/up-down counter:** active duration/in-flight work under supported model.

Never average precomputed percentiles across instances. Histograms can aggregate when bucket boundaries/model align. Client-side quantiles have different aggregation properties. Choose buckets around SLO and diagnostic thresholds.

Cardinality budget

Time-series count roughly multiplies distinct label values:

TEXT

```
routes(40) * methods(5) * outcomes(8) * regions(4)
= 6,400 potential series before instances/status/etc.
```

Adding `customerId` with one million values explodes cost. Good dimensions are bounded: route template, operation, method, outcome class, dependency, region, version. Put high-cardinality request/order/event IDs in logs or sampled traces.

Metric names and units are an API. `request.duration` must define seconds/milliseconds through the telemetry convention, start/stop boundary, error/cancellation classification, and route fallback. Missing labels and renamed routes can break dashboards silently.

4. RED and USE

RED for request/event-driven components

- **Rate:** requests/records per second;
- **Errors:** failures under the operation contract, separated from expected client outcomes;
- **Duration:** latency distribution, including queue time as defined.

Add in-flight and rejection because rate alone does not reveal queue/saturation. For Kafka, use input rate, processing failure, processing/end-to-end age, consumer lag, retry/DLQ, and rebalance.

USE for resources

- **Utilization:** fraction/time resource is busy;
- **Saturation:** queued/waiting work beyond immediate service capacity;
- **Errors:** resource failures.

Examples:

Resource	Utilization	Saturation	Errors
CPU	busy time	runnable queue/throttling	machine/runtime counters
DB pool	active/max	acquisition waiters/time	timeout/validation failures
disk	busy/throughput	queue latency/depth	I/O errors
executor	active capacity	queue depth/oldest age	rejection/task failure
heap/GC	occupancy/allocation context	allocation pressure/pause symptoms	OOM/allocation failure

Interpret OS/JVM/container metrics using platform documentation. CPU "100%" may mean one core or quota depending on metric. Heap occupancy alone is not a memory leak proof.

5. Micrometer and OpenTelemetry mental model

Micrometer

Micrometer provides instrumentation abstractions for meters and observations, with registries/handlers exporting to monitoring systems through supported modules. Spring Boot can auto-configure integrations. Application code should record semantic operation metrics, not depend on a vendor query language.

Dependency-requiring example:

JAVA

```java
final class InventoryClient {
    private final MeterRegistry registry;

    InventoryResult reserve(Command command) {
        Timer.Sample sample = Timer.start(registry);
        String outcome = "success";
        try {
            return call(command);
        } catch (RuntimeException failure) {
            outcome = classify(failure); // bounded vocabulary
            throw failure;
        } finally {
            sample.stop(Timer.builder("inventory.request")
                    .tag("operation", "reserve")
                    .tag("outcome", outcome)
                    .register(registry));
        }
    }
}
```

Avoid registering meters with an unbounded tag on every call. Prefer framework observation conventions when they already capture request/client boundaries, and customize bounded fields.

OpenTelemetry

OpenTelemetry defines APIs/SDK concepts for traces, metrics, logs, context propagation, semantic conventions, sampling, and OTLP export. A trace contains spans with parent/link relationships, attributes, events, status, and timing. Messaging may use span links when causal work is not a strict synchronous child.

Key boundaries:

- instrumentation API records telemetry without choosing backend;
- SDK/processors/samplers/exporters control collection/export;
- context propagators encode/decode cross-process context;
- collector can receive, process, sample, route, and export;
- backend stores/queries/visualizes.

Do not put secrets/PII or high-cardinality payloads into span attributes. Head sampling decides near trace start and may miss rare failures; tail sampling can use completed trace signals but requires collector buffering/capacity. Sampling changes diagnostic coverage; metrics remain the primary aggregate SLI source.

Instrumentation must fail safely. A slow exporter must not block the request indefinitely; bounded queues can drop telemetry under overload. Measure dropped spans/logs/metrics so "no evidence" is not mistaken for health.

6. SLI, SLO, and error budget

Formal definitions

An SLI is a measured ratio or distribution tied to user experience. For request availability:

```text
SLI = good eligible requests / total eligible requests
```

Define:

- service and user journey;
- eligible population and exclusions;
- good-event rule (status plus latency threshold);
- measurement point (load balancer, service, synthetic, client);
- rolling/calendar window and data completeness;
- treatment of retries, cancellations, rate-limited abuse, planned maintenance, and missing telemetry.

Example: "Over a rolling 30 days, 99.9% of eligible `CreateOrder` requests accepted by the service boundary complete within 750 ms without a server-attributable terminal failure." This still needs exact outcome mapping.

For target S over event count N, error budget is $(1-S)N$. For time-style intuition, 99.9% over 30 days corresponds to about 43.2 minutes, but request-based SLO budget depends on traffic and is usually more faithful for request services.

Burn rate

Burn rate compares observed bad-event fraction to allowed bad fraction:

```text
burn = observedErrorRate / (1 - SLO)
```

For 99.9%, allowed fraction is 0.001. Observed 1% bad gives burn 10. At sustained burn 10, a 30-day budget is consumed in about 3 days. Multi-window alerts combine a fast window with a longer confirmation window: high burn catches severe incidents quickly; lower sustained burn catches slow erosion. Exact thresholds/windows follow organizational policy and traffic volume.

Avoid alerting directly on CPU absent user risk. Page on SLO burn or a condition likely to become imminent user impact; create tickets/dashboard signals for lower urgency. Every alert needs owner, runbook, and tested routing.

7. Operational diagnostic method

Evidence loop

1. state user impact, scope, start time, and SLI change;
2. stabilize: freeze risky deploys, shed/degrade, rollback or isolate when evidence supports it;
3. compare recent changes: deployment, config, traffic, dependency, schema, partition, infrastructure;
4. use RED to locate affected operation/dependency;
5. use USE to test saturation hypotheses;
6. inspect traces/logs for representative failing paths;
7. collect JVM/database/broker evidence without destructive load;
8. change one reversible factor when possible;
9. verify SLI recovery and absence of displaced failure;
10. preserve timeline/evidence and define prevention with owner/date.

Worked incident

Symptom: create-order p99 jumps from 300 ms to 4 s; error rate remains low, DB pool wait rises, CPU normal.

Hypotheses:

- query/lock duration increased;
- connections leaked/held during remote calls;
- pool/database capacity changed;
- traffic mix shifted to slow path.

Evidence:

- trace shows payment HTTP span occurs inside transaction span;
- pool active=max and wait p99 3 s;
- DB execution itself 40 ms; lock wait low;
- deploy introduced fraud call within transactional method.

Mitigation: roll back or move remote call outside local transaction using a durable workflow, then verify pool wait and request SLI. Prevention: transaction-duration metric, architecture test/review rule, integration load test, explicit workflow state.

Do not "fix" by only enlarging the pool; that can transfer saturation to the database and retain the flawed boundary.

TRY IT | 8. Interview questions and model checkpoints

Q1. Metrics versus logs versus traces?

Model checkpoint: metrics reveal aggregate rate/error/duration/saturation and alert; logs capture discrete structured events; traces connect sampled causal paths. Use together and respect cardinality/privacy/cost.

Q2. What makes an SLI defensible?

Model checkpoint: user journey, eligible population, good rule, measurement point, window, exclusions, retry/cancellation policy, and data-quality monitoring.

Q3. Why not tag metrics with order ID?

Model checkpoint: unbounded cardinality creates a series per value and cost/memory failure. Put IDs in controlled logs/traces; metrics use bounded route/outcome/dependency dimensions.

Q4. RED versus USE?

Model checkpoint: RED starts from service operations (rate/errors/duration); USE starts from resources (utilization/saturation/errors). Correlate user impact with bottleneck evidence.

SDE-2 follow-ups

1. Design telemetry for a saga spanning HTTP, outbox, Kafka, and three consumers.
2. A collector loses 40% of spans during overload. Explain how metrics still detect impact and how telemetry loss is surfaced.
3. Define a 99.95% latency/availability SLO with low-traffic statistical considerations.
4. Build a burn-rate alert policy and explain false positives from client errors/retries.

TRY IT | 9. Exercises

1. Audit a metric namespace and calculate worst-case time-series cardinality.
2. Convert five prose logs into structured events and redact sensitive fields.
3. Write RED/USE dashboards for HTTP, DB pool, Kafka consumer, Redis, and JVM.
4. Define availability and freshness SLIs for a catalog served from cache.
5. Run a paper incident: p99 latency, pool saturation, Kafka lag, and normal CPU. Produce hypotheses and discriminating evidence.

RECAP | 10. Summary checklist

- [ ] Telemetry begins with user and operational questions.
- [ ] Logs are structured, correlated, redacted, and not duplicated at every layer.
- [ ] Metrics have correct types, units, boundaries, and bounded labels.
- [ ] Traces propagate context safely and sampling/export loss is observable.
- [ ] RED and USE connect impact to capacity evidence.
- [ ] SLI population, good rule, source, window, exclusions, and quality are explicit.
- [ ] Alerts use error-budget urgency and have owners/runbooks.
- [ ] Incident actions are evidence-based, reversible, and verified against the SLI.

Primary references

- OpenTelemetry Specification: https://opentelemetry.io/docs/specs/
- OpenTelemetry Java Documentation: https://opentelemetry.io/docs/languages/java/
- Micrometer Documentation: https://docs.micrometer.io/micrometer/reference/
- Spring Boot Actuator Observability Reference: https://docs.spring.io/spring-boot/reference/actuator/observability.html
- Google SRE Workbook, "Alerting on SLOs": https://sre.google/workcontent/master/alerting-on-slos/

> **Version boundary:** semantic-convention names, Micrometer observation APIs, Spring auto-instrumentation, exporter behavior, and OpenTelemetry SDK defaults evolve. Pin and test the versions managed by the application. The conceptual SLI/SLO and cardinality contracts are version-independent; code baseline is Java 21.

SDE-2 CORE CHAPTER

Chapter 5 - A Rigorous HLD Interview Method with Worked Java-Backend Cases

Learning objectives

After this chapter, you should be able to:

- run a 45-60 minute high-level design interview as a sequence of explicit decisions;
- turn vague scale claims into unit-bearing estimates and bottleneck hypotheses;
- define APIs, data models, partitioning, consistency, and failure behavior before drawing a large architecture;
- connect Java implementation boundaries to connection pools, threads/virtual threads, serialization, GC, and graceful shutdown;
- work through a URL-shortening service and multi-channel notification platform; and
- answer SDE-2 design follow-ups with invariants, tradeoffs, evidence, and evolution paths.

1. The interview operating system

Phase 1: frame requirements and invariants (5-8 minutes)

Ask questions that change architecture:

- primary user journeys and explicit non-goals;
- read/write ratio, payload size, retention, peak and geographic distribution;
- availability, latency, durability, and consistency requirements per operation;
- identity, ordering, uniqueness, privacy, residency, and deletion;
- synchronous versus asynchronous outcome;
- abuse/rate limits and multi-tenancy;
- existing constraints: Java stack, relational database, Kafka, cloud, team size.

Write the hardest invariants:

```text
one idempotency key -> one logical command result
short code -> at most one active destination within namespace
one notification request -> no more than policy-allowed deliveries per channel
tenant A cannot read or consume tenant B's capacity/data
```

Do not spend half the interview collecting trivia. State reasonable assumptions and invite correction.

Phase 2: estimate scale (4-6 minutes)

Calculate average and peak RPS, concurrent work, storage/year, bandwidth, cache working set, partitions, and downstream rate. Show equations and ranges. Use estimates to justify choices, not to perform fake precision.

Sensitivity matters more than one number:

```text
baseline peak 10k RPS
launch/incident factor 3 -> 30k
cache miss rises from 1% to 30% -> DB fallback from 100 to 3,000 RPS
```

Phase 3: contracts and data model (7-10 minutes)

Define:

- APIs/commands/events, idempotency and pagination;
- statuses and error/unknown outcomes;
- primary identities and unique constraints;
- access patterns and candidate indexes;
- authoritative source versus cache/derived views;
- event schema/key/order and retention.

An API is not just endpoint names. Include method semantics, operation identity, conditional update/version, max size, and asynchronous status.

Phase 4: minimum viable architecture (8-10 minutes)

Draw the shortest path satisfying current numbers:

```text
edge/admission -> stateless Java service -> authoritative store
                                  |
                                  +-> outbox/broker -> workers
                                  +-> cache/read model where justified
```

Label ownership and failure domains. Avoid adding five databases before identifying why one relational database plus replicas/partitions fails.

Phase 5: deep dive on two risks (12-18 minutes)

Choose the most relevant:

- hot key/partition and rebalancing;
- local versus distributed transaction and duplicates;
- cache consistency/stampede;
- Kafka ordering/rebalance/retry;
- multi-region consistency/failover;
- rate limiting/backpressure;
- schema/data migration;
- security/privacy;
- observability/SLO and incident recovery.

Walk one happy path and at least three failure paths. A strong answer names the state after timeout and how recovery discovers truth.

Phase 6: operations, evolution, and summary (5 minutes)

Close with:

- RED/USE metrics, SLI/SLO, probes, logs/traces, backlog age;
- deployment/schema compatibility and rollback;
- capacity/admission and graceful shutdown;
- top tradeoffs and next scaling trigger;
- concise architecture recap tied to requirements.

2. Decision record template

For each major choice, say:

TEXT

```
Decision: key URL mappings by short-code hash.
Reason: point reads dominate and name the code.
Guarantee: one code maps to one active destination under a unique constraint.
Cost: range/admin queries fan out or need a secondary index.
Failure: hot viral code creates read hotspot.
Evidence/trigger: per-key traffic and shard saturation; add edge/cache replication.
Alternative: range partition by creation time rejected because redirect lookup lacks time.
```

This pattern demonstrates judgment. "Use Cassandra because scale" does not.

3. Worked case A: URL shortening and redirect service

Requirements

- Create a short URL for a valid `https` destination.
- Redirect by code with p99 under 50 ms in primary regions.
- 100 million new links/month; 100:1 redirect-to-create ratio; peak redirect 80k RPS.
- Default links do not expire; owners can disable them.
- Custom aliases are unique within tenant namespace.
- Analytics can lag minutes and must not slow redirects.
- Malicious destinations/abuse need policy; deletion/privacy supported.

Non-goals: exact real-time global click count, guaranteed arbitrary custom word, browser-side preview rendering.

Estimate

100 million/month is about 39 writes/s average; assume 10x peak ≈ 400/s. If one mapping record averages 600 bytes including indexes/overhead estimate range, raw annual new data ≈ 720 GB before replication/backups; validate actual encoding. Redirect 80k RPS at 1 KiB response headers/body average ≈ 78 MiB/s before edge/CDN overhead. A cache working set depends on popularity skew; measure top-code coverage.

API

TEXT

```
POST /v1/links
Idempotency-Key: ...
{destination, customAlias?, expiresAt?}
-> 201 {code, shortUrl, version, createdAt}

GET /{code}
-> 302 or 307/308 according to product permanence/cache policy

DELETE /v1/links/{code}
If-Match: ...
-> 204 / 412 / 404
```

Redirect status choice affects client/cache permanence; do not use permanent caching while owners can edit/disable unless policy reconciles it. Validate destination scheme, canonicalization, length, and block unsupported internal schemes. Avoid fetching arbitrary destination URLs synchronously in the create path because it can create SSRF; a separate safe scanner needs network egress controls.

Data and ID generation

Authoritative mapping:

```text
Link(code PK, tenantId, destination, status, ownerId,
     createdAt, expiresAt, version, abuseState)
unique(tenantId, customAlias) when custom namespace applies
```

Generated code options:

- encode a unique numeric ID in base62: compact/collision-free under ID service, but exposes sequence/volume and needs globally available allocation;
- generate random bits and base62: decentralized/unpredictable, but insert must retry rare unique collision;
- hash destination: same destination may need different owners/policies; collision handling and predictability;
- pre-generated code pool: absorbs generator outage but adds inventory/security/operations.

Choose 72-96 random bits encoded in URL-safe alphabet for unpredictability and collision margin, with database unique constraint as final guard. Exact length follows threat/capacity calculation. Custom alias uses normalized namespace plus constraint.

Partition and read path

Partition by hash of `code` because redirects name it and traffic spreads by code count. One viral code is still a hot key. Read flow:

1. edge rate/abuse control;
2. regional cache/near-cache lookup by versioned code;
3. on miss, mapping store point read;
4. validate active/not expired/abuse policy;
5. cache bounded projection with TTL and invalidation/version;
6. return redirect;
7. emit click telemetry asynchronously under sampling/privacy policy.

For a viral code, replicate at CDN/edge or local cache. Mapping changes must invalidate/bump version and account for cached redirect status. Checkout-style strong correctness is not needed for analytics, but disable/abuse response may require short cache TTL or push invalidation.

Create path and consistency

1. authenticate owner and enforce quota;
2. reserve idempotency key/payload fingerprint;
3. validate/canonicalize destination without unsafe fetch;
4. generate candidate code;
5. insert mapping and completed key under transaction; retry unique random collision, not arbitrary DB failure;
6. append `LinkCreated` outbox event for analytics/scanning;
7. commit and respond.

The create response is read-your-writes because it returns committed record. A subsequent regional redirect may route to a lagging replica; route initial reads to primary, populate cache on write, or carry version token if the product requires immediate global visibility.

Analytics

Redirect nodes produce compact `LinkClicked` events keyed based on required aggregation/order-not necessarily code if hot keys would overload one partition. Approximate/counting aggregation can shard `(code,bucket)` and merge. Protect redirect latency: bounded nonblocking emission, local buffer only if durability policy allows, or broker client with strict capacity. Define what lost/sampled clicks mean.

Failures and operations

Failure	Behavior
cache unavailable	bounded fallback to store; local admission prevents store collapse
mapping replica lag	session/version or primary route for just-created code; otherwise documented delay
code store shard unavailable	redirects for that shard degrade/fail; replicas/failover with fencing
outbox/broker down	redirects continue; click analytics may buffer/drop under stated policy; mapping outbox backlogs
abusive viral code	edge block/rate policy, rapid invalidation, audit
cache stampede	single-flight, stale policy for active links, source bulkhead, edge cache

SLIs: eligible redirect success and latency; create availability; disable propagation freshness; mapping-store/caching errors; analytics freshness separately. Track hot keys without using code as metric label.

Java implementation notes

Use immutable DTOs/records, bounded validation, `SecureRandom` through an owned code generator, JDBC/JPA projection reads, and explicit `Clock`. Virtual threads can support blocking redirect lookups but store/cache connections remain bounded. Avoid allocating large URI/JSON graphs on the hot redirect path. Profile before micro-optimizing base62.

Case checkpoint

A complete answer mentions code uniqueness, tenant namespace, read hotspot, cache invalidation/disable freshness, idempotent create, store partition, analytics decoupling, SSRF/abuse, failover, and SLO-not merely "Redis plus database."

4. Worked case B: multi-channel notification platform

Requirements

- Services submit email, SMS, and push notifications.
- Peak 20k requests/s; campaigns can burst millions of recipients.
- Transactional notifications are high priority with 99% dispatched within 30 seconds.
- Marketing can be delayed and must honor consent/quiet hours/unsubscribe.
- Provider limits and costs differ by channel/tenant/region.
- Request retries must not create unintended duplicate delivery.
- Templates and destination data are sensitive; audit and deletion policies apply.

Clarify that "exactly once notification" is not fully controllable: a provider can accept and deliver while its acknowledgement is lost, and email/SMS recipients can receive duplicates. The platform promises deduplicated intent and best available provider idempotency/reconciliation.

API and state machine

TEXT

```
POST /v1/notifications
Idempotency-Key: ...
{tenantId derived/authenticated, recipientRef, templateId,
 channelPolicy, variables, priority, scheduleAt}
-> 202 {notificationId, statusUrl}

GET /v1/notifications/{id}
-> ACCEPTED | SCHEDULED | DISPATCHING | SENT | DELIVERED?
 | RETRY_WAIT | FAILED_FINAL | CANCELLED | OUTCOME_UNKNOWN
```

Separate `SENT_TO_PROVIDER` from provider delivery/read signals. Webhooks are untrusted input: verify signature, replay window, provider event ID, and state transition. `DELIVERED` semantics differ by channel/provider.

Architecture

TEXT

```
API -> relational command/state + outbox
              |
              v
        scheduler/router -> priority/channel Kafka topics
                                  |
              +-------------------+-----------------+
              v                   v                 v
         email workers       SMS workers        push workers
              |                   |                 |
         provider pool        provider pool       APNs/etc.

webhook ingress -> inbox/dedupe -> notification state
consent/template services/read models
rate-limit and provider-health control plane
```

The API transaction stores notification intent, idempotency outcome, and outbox. It returns `202` because delivery is asynchronous. Workers are separate bulkheads by channel/provider/priority.

Data model

TEXT

```
Notification(id, tenantId, recipientRef, templateVersion,
             channel, priority, scheduleAt, state, attempt,
             operationKey, sourceVersion, createdAt, updatedAt)
DeliveryAttempt(id, notificationId, provider, providerOperationId,
                state, startedAt, deadline, responseCode, nextAttemptAt)
Inbox(provider, providerEventId unique, receivedAt)
Outbox(eventId, aggregateId, type, payloadRef, occurredAt, publishedAt)
```

Minimize destination/content in durable event payloads. Workers can retrieve encrypted/authorized rendering data by reference, but that adds dependency/failure; or use encrypted payload with strict key/retention. Templates are immutable/versioned so a queued notification does not silently change wording.

Partitioning and ordering

Key events by `notificationId` for per-notification state order. Campaign generation should not emit millions synchronously in one API transaction. Store a campaign job and page recipients under snapshot/consent semantics, generating bounded notification batches.

One tenant campaign can monopolize partitions/providers. Apply hierarchical admission:

- global provider/channel rate;
- tenant quota and fair scheduling;
- priority reservation for transactional traffic;
- per-destination anti-abuse limit;
- worker concurrency and bounded retry queue.

Strict priority can starve marketing; use weighted fair queues/reserved capacity. Rate tokens can be leased locally from a central policy to reduce hot coordination, accepting bounded imprecision.

Worker flow

1. consumer receives notification event at least once;
2. DB transaction checks inbox/event ID and current notification state/version;
3. reserve a delivery attempt with stable `providerOperationId` and transition to `DISPATCHING`;
4. commit; remote provider call occurs outside DB transaction with deadline;
5. transaction records accepted/rejected/unknown result and outbox state event;
6. commit Kafka offset after durable state;
7. webhook/reconciliation can advance state idempotently.

If crash occurs after provider accepted but before result record, replay reads `DISPATCHING/unknown` and queries provider by operation ID or safely repeats only if provider deduplicates. Otherwise route to reconciliation instead of guaranteed duplicate.

Retry and DLQ

- `429`/temporary provider outage: reschedule with server guidance, jitter, deadline/campaign expiry, and rate control;
- invalid destination/template: terminal and feed hygiene signal;
- authentication/config failure: open breaker/stop provider path and page owner;
- unknown outcome: reconcile before sending anew;
- poison event/code bug: quarantine with secure reference, not repeated hot loop;
- retry topic preserves key or state version prevents stale overwrite.

Campaign notifications past their relevance deadline should expire rather than be delivered days late. Transactional notifications can fail over provider if consent/channel policy and provider operation identity support it. Provider failover can itself duplicate if original outcome is unknown.

Consent and security

Check consent at acceptance and again near dispatch for marketing because queue delay matters. Store the policy/version evidence used. Unsubscribe must propagate within a defined freshness SLO. Tenant-scoped authorization protects status APIs. Template variables are untrusted content; escape for HTML/text/channel and prevent template injection. Restrict preview/test sends.

Encrypt sensitive destinations at rest under managed keys, tokenize/reference where possible, redact logs/traces, and define retention/deletion across DB, events, DLQ, provider, and backups. Rate limiting is also abuse/cost control.

Observability and SLO

Metrics:

- accepted/dispatched/provider-accepted/delivery-callback rate by bounded tenant tier/channel/provider/outcome;
- queue oldest age and count by priority/channel;
- end-to-end acceptance-to-dispatch histogram;
- provider latency/error/rate-limit/breaker;
- retries, unknown outcome, reconciliation age, DLQ age;
- consent suppression, duplicate/inbox hit;
- consumer lag/rebalance and DB pool/saturation.

Trace a sample from API to outbox/Kafka/worker/provider using links/context, but do not put phone/email/content in attributes. SLOs differ: transactional dispatch freshness, marketing completion deadline, consent propagation, and status API availability.

Case checkpoint

A strong design separates intent, attempt, provider acceptance, and delivery; uses durable workflow/outbox/inbox; scopes ordering; protects priority and provider capacity; treats unknown outcome/reconciliation explicitly; and includes consent, privacy, cost, and operational backlog.

5. Cross-case comparison

Decision	URL shortener	Notifications
dominant path	low-latency read	asynchronous workflow
authoritative key	short code	notification ID/idempotency key
partition pressure	viral read hot key	tenant campaign/provider limits
consistency focus	code uniqueness and disable freshness	state transition, deduped intent, unknown remote outcome
cache	central redirect optimization	limited config/template/consent views with freshness rules
Kafka	analytics/invalidation	core workflow transport
hard failure	store/cache outage and stale redirect	provider timeout, retry, duplicate delivery, backlog
security	malicious URL/SSRF/abuse	consent, content injection, PII, provider webhook

The architecture follows requirements; no universal "system-design stack" exists.

TRY IT | 6. HLD question bank and model checkpoints

1. **Where is the source of truth?**
   Checkpoint: identify authoritative store per fact and every derived cache/index/event view plus rebuild/repair.

2. **What is the partition key?**
   Checkpoint: common routing, load skew, hot key, transaction/order scope, rebalancing, secondary fan-out.

3. **What happens on timeout after a side effect?**
   Checkpoint: unknown outcome, stable operation ID, status/reconciliation, no assumption of rollback.

4. **How do you prevent duplicates?**
   Checkpoint: API idempotency, outbox event ID, consumer inbox/state predicate, provider idempotency; name retention.

5. **What consistency does the user observe?**
   Checkpoint: operation-specific linearizable/session/bounded/eventual contract and failover behavior.

6. **How do you handle a hot key?**
   Checkpoint: measure traffic, cache/replicate reads, split if semantics allow, isolate tenant, admission; adding random salt costs order/locality.

7. **How does the system degrade?**
   Checkpoint: explicit priority, stale-safe responses, early rejection, queue age/limit, core path isolation.

8. **Why Kafka?**
   Checkpoint: durable backlog, independent consumers, partition order/parallelism, replay; accept latency, duplicates, operations, schema burden. Do not use merely because "microservices."

9. **How do you deploy schema change?**
   Checkpoint: additive/expand-migrate-contract, mixed versions, backfill, compatibility tests, rollback/repair.

10. **How do you know it works?**
    Checkpoint: SLIs/SLOs, RED/USE, plans/load/chaos tests, reconciliation, data quality, runbooks.

11. **Where does Java matter?**
    Checkpoint: transaction/proxy boundary, connection/executor limits, virtual threads not resource magic, serialization compatibility, GC/allocation evidence, context propagation, graceful shutdown.

12. **When would you change the design?**
    Checkpoint: measurable trigger-store capacity, shard size, hot-key rate, SLO burn, cache working set, backlog recovery time-not speculative scale.

7. Final SDE-2 interview checklist

- [ ] Requirements and non-goals are explicit.
- [ ] Invariants, identities, and operation-specific consistency are named.
- [ ] Estimates include units, peaks, retention, replication, and headroom.
- [ ] APIs include idempotency, bounds, versions, errors, and async status.
- [ ] Data model and indexes derive from access patterns.
- [ ] Minimal architecture labels ownership and failure domains.
- [ ] Partitioning covers skew, hot keys, ordering, fan-out, and rebalance.
- [ ] Cross-system effects have outbox/inbox/idempotency and unknown-outcome recovery.
- [ ] Cache and queues have staleness/capacity/failure contracts.
- [ ] Security, privacy, abuse, and tenant isolation are first-class.
- [ ] Observability connects SLIs to RED/USE and repair backlogs.
- [ ] Evolution uses measurable triggers and compatible migrations.
- [ ] Summary states decisions, costs, alternatives, and next bottleneck.

TRY IT | 8. Exercises and mock loops

1. Re-run the URL shortener assuming editable destinations require global disable within two seconds. Change cache/replication design.
2. Re-run notifications with no Kafka allowed. Design a database queue and compare claim/lock, scale, replay, and operations.
3. Design a distributed rate limiter for 100k tenants with a few whales; quantify precision versus hot coordination.
4. Design a job scheduler with delayed execution, leases/fencing, retries, cancellation, and tenant fairness.
5. Design a file-processing service with multi-part upload, virus scanning, idempotency, lifecycle, and regional storage.
6. Conduct a 45-minute mock. Spend at most 8 minutes clarifying, show three estimates, deep-dive two failures, and reserve 5 minutes to summarize.

9. Common HLD anti-patterns

- **Boxes before contracts:** drawing gateway, cache, Kafka, and five stores without a user-visible invariant or failure behavior.
- **Scale adjectives without arithmetic:** "billions" with no peak RPS, bytes, retention, or hot-key distribution.
- **Exactly-once by assertion:** ignoring API retries, outbox publish/mark, consumer effect/offset, and remote provider unknown outcomes.
- **Cache as correctness:** treating a miss, stale value, or eviction as impossible and omitting source-collapse recovery.
- **Partitioning by record count:** ignoring one whale tenant, viral key, expensive event, or cross-shard transaction.
- **Retry as availability:** amplifying a saturated dependency and repeating non-idempotent work.
- **Multi-region by arrows:** no write authority, quorum, fencing, read-your-writes, conflict, or residency contract.
- **Observability appendix:** dashboards listed after design with no SLI, backlog age, reconciliation, or capacity signal.
- **Premature exotic technology:** paying operational complexity before a measured trigger.
- **Java thread optimism:** increasing platform/virtual threads while connections, CPU, memory, or downstream concurrency remain fixed.

Repair an answer by choosing one critical operation and narrating: request identity, route, validation, authoritative read/write, commit acknowledgement, derived event/cache, response, timeout state, retry identity, and telemetry. Then scale the measured bottleneck. This execution thread turns a diagram into an engineering design.

Primary references

- Java SE 21 Documentation: https://docs.oracle.com/en/java/javase/21/
- Apache Kafka Documentation: https://kafka.apache.org/documentation/
- OpenTelemetry Specification: https://opentelemetry.io/docs/specs/
- RFC 9110, "HTTP Semantics": https://www.rfc-editor.org/rfc/rfc9110
- OWASP Application Security Verification Standard: https://owasp.org/www-project-application-security-verification-standard/

> **Version boundary:** worked designs are technology-neutral at the contract layer. Kafka, databases, caches, Spring integrations, cloud load balancers, and Java runtimes must be verified at selected versions. Java 21 is the implementation baseline; later JDK structured-concurrency or framework conveniences are optional deltas, not assumed guarantees.

SDE-2 CORE CHAPTER

Chapter 6 - Executable Distributed-Systems Model

This dependency-free Java 21 model makes partitioning, offset processing, deadlines, retry budgets, circuit state, and SLO calculations executable.

JAVA (continued 1/12)

```java
import java.time.Duration;
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
import java.util.Objects;
import java.util.OptionalLong;
import java.util.Set;
import java.util.SplittableRandom;
import java.util.concurrent.TimeUnit;

/**
 * Dependency-free Java 21 executable models for capacity, token-bucket
 * admission, retry budgets, Kafka-style contiguous offsets, and saga state.
 */
public final class DistributedSystemsPatterns {
    private DistributedSystemsPatterns() {
    }

    public record CapacityEstimate(
            double averageRps,
            double peakRps,
            double concurrentDependencyCalls,
            double gibPerDay,
            double replicatedTibPerYear) {
    }

    public static CapacityEstimate estimate(
            long requestsPerDay,
            double peakFactor,
            double dependencyFraction,
            Duration dependencyLatency,
            long eventsPerDay,
            long bytesPerEvent,
```

JAVA (continued 2/12)

```
            int replicationFactor) {
        if (requestsPerDay < 0 || eventsPerDay < 0 || bytesPerEvent < 0) {
            throw new IllegalArgumentException("counts must be nonnegative");
        }
        requirePositiveFinite(peakFactor, "peakFactor");
        if (dependencyFraction < 0 || dependencyFraction > 1) {
            throw new IllegalArgumentException(
                    "dependencyFraction must be in [0,1]");
        }
        Objects.requireNonNull(dependencyLatency, "dependencyLatency");
        if (dependencyLatency.isNegative()) {
            throw new IllegalArgumentException("negative dependency latency");
        }
        if (replicationFactor < 1) {
            throw new IllegalArgumentException("replicationFactor must be >= 1");
        }

        double averageRps = requestsPerDay / 86_400.0;
        double peakRps = averageRps * peakFactor;
        double seconds = dependencyLatency.toNanos() / 1_000_000_000.0;
        double concurrent = peakRps * dependencyFraction * seconds;
        double bytesPerDay = Math.multiplyExact(eventsPerDay, bytesPerEvent);
        double gibPerDay = bytesPerDay / Math.pow(1024.0, 3);
        double replicatedTibPerYear = bytesPerDay * 365.0
                * replicationFactor / Math.pow(1024.0, 4);
        return new CapacityEstimate(averageRps, peakRps, concurrent,
                gibPerDay, replicatedTibPerYear);
    }

    /** Monotonic-time token bucket supporting fractional refill. */
    public static final class TokenBucket {
        private final double capacity;
        private final double refillPerNano;
        private double tokens;
```

JAVA (continued 3/12)

```
        private long lastNanos;

        public TokenBucket(double capacity, double refillPerSecond,
                long initialNanos) {
            requirePositiveFinite(capacity, "capacity");
            requirePositiveFinite(refillPerSecond, "refillPerSecond");
            this.capacity = capacity;
            this.refillPerNano = refillPerSecond / 1_000_000_000.0;
            this.tokens = capacity;
            this.lastNanos = initialNanos;
        }

        public synchronized boolean tryAcquire(double cost, long nowNanos) {
            requirePositiveFinite(cost, "cost");
            if (cost > capacity) {
                return false;
            }
            refill(nowNanos);
            if (tokens + 1e-12 < cost) {
                return false;
            }
            tokens -= cost;
            return true;
        }

        public synchronized double available(long nowNanos) {
            refill(nowNanos);
            return tokens;
        }

        private void refill(long nowNanos) {
            if (nowNanos < lastNanos) {
                throw new IllegalArgumentException("monotonic time moved backward");
            }
```

JAVA (continued 4/12)

```
            long elapsed = nowNanos - lastNanos;
            tokens = Math.min(capacity, tokens + elapsed * refillPerNano);
            lastNanos = nowNanos;
        }
    }

    public record RetryAttempt(int number, Duration delay,
                               Duration remainingAfterDelay) {
    }

    /**
     * Builds bounded full-jitter delays. Processing time is charged before
     * each optional retry; no delay is returned once the deadline is spent.
     */
    public static List<RetryAttempt> retryPlan(
            int maxAttempts,
            Duration totalBudget,
            Duration perAttemptCost,
            Duration baseBackoff,
            Duration maxBackoff,
            long randomSeed) {
        if (maxAttempts < 1) {
            throw new IllegalArgumentException("maxAttempts must be >= 1");
        }
        requirePositive(totalBudget, "totalBudget");
        requirePositive(perAttemptCost, "perAttemptCost");
        requirePositive(baseBackoff, "baseBackoff");
        requirePositive(maxBackoff, "maxBackoff");

        long remaining = totalBudget.toNanos();
        long attemptCost = perAttemptCost.toNanos();
        long base = baseBackoff.toNanos();
        long max = maxBackoff.toNanos();
```

JAVA (continued 5/12)

```
        var random = new SplittableRandom(randomSeed);
        var plan = new ArrayList<RetryAttempt>();

        for (int attempt = 1; attempt <= maxAttempts; attempt++) {
            if (remaining < attemptCost) {
                break;
            }
            remaining -= attemptCost;
            if (attempt == maxAttempts) {
                break;
            }
            int shift = Math.min(attempt - 1, 62);
            long exponential;
            try {
                exponential = Math.multiplyExact(base, 1L << shift);
            } catch (ArithmeticException overflow) {
                exponential = Long.MAX_VALUE;
            }
            long cap = Math.min(max, exponential);
            long delay = cap == Long.MAX_VALUE
                    ? random.nextLong(Long.MAX_VALUE)
                    : random.nextLong(cap + 1);
            if (delay >= remaining) {
                break;
            }
            remaining -= delay;
            plan.add(new RetryAttempt(attempt + 1,
                    Duration.ofNanos(delay), Duration.ofNanos(remaining)));
        }
        return List.copyOf(plan);
    }

    /**
```

JAVA (continued 6/12)

```
     * Tracks completed offsets and exposes the next safe commit offset: the
     * first offset not yet completed. Useful when processing finishes out of
     * order while commits must advance only across a contiguous prefix.
     */
    public static final class ContiguousOffsetTracker {
        private long nextCommitOffset;
        private final Set<Long> completedOutOfOrder = new HashSet<>();

        public ContiguousOffsetTracker(long startingOffset) {
            if (startingOffset < 0) {
                throw new IllegalArgumentException("negative starting offset");
            }
            this.nextCommitOffset = startingOffset;
        }

        public synchronized OptionalLong complete(long offset) {
            if (offset < nextCommitOffset) {
                return OptionalLong.empty(); // duplicate completion
            }
            completedOutOfOrder.add(offset);
            long before = nextCommitOffset;
            while (completedOutOfOrder.remove(nextCommitOffset)) {
                nextCommitOffset++;
            }
            return nextCommitOffset == before
                    ? OptionalLong.empty()
                    : OptionalLong.of(nextCommitOffset);
        }

        public synchronized long nextCommitOffset() {
            return nextCommitOffset;
        }
```

JAVA (continued 7/12)

```
        public synchronized Set<Long> gapsSnapshot() {
            return Collections.unmodifiableSet(
                    new HashSet<>(completedOutOfOrder));
        }
    }

    public enum SagaState {
        PENDING,
        INVENTORY_RESERVED,
        PAYMENT_UNKNOWN,
        PAYMENT_AUTHORIZED,
        CONFIRMED,
        COMPENSATING,
        CANCELLED,
        FAILED_RECONCILIATION
    }

    public enum SagaEvent {
        INVENTORY_RESERVED,
        PAYMENT_TIMED_OUT,
        PAYMENT_CONFIRMED,
        CONFIRM_ORDER,
        CANCEL_REQUESTED,
        COMPENSATION_COMPLETED,
        COMPENSATION_FAILED
    }

    public static SagaState transition(SagaState state, SagaEvent event) {
        Objects.requireNonNull(state, "state");
        Objects.requireNonNull(event, "event");
        return switch (state) {
            case PENDING -> switch (event) {
                case INVENTORY_RESERVED -> SagaState.INVENTORY_RESERVED;
```

JAVA (continued 8/12)

```
                case CANCEL_REQUESTED -> SagaState.CANCELLED;
                default -> invalid(state, event);
            };
            case INVENTORY_RESERVED -> switch (event) {
                case PAYMENT_TIMED_OUT -> SagaState.PAYMENT_UNKNOWN;
                case PAYMENT_CONFIRMED -> SagaState.PAYMENT_AUTHORIZED;
                case CANCEL_REQUESTED -> SagaState.COMPENSATING;
                default -> invalid(state, event);
            };
            case PAYMENT_UNKNOWN -> switch (event) {
                case PAYMENT_CONFIRMED -> SagaState.PAYMENT_AUTHORIZED;
                case CANCEL_REQUESTED -> SagaState.COMPENSATING;
                default -> invalid(state, event);
            };
            case PAYMENT_AUTHORIZED -> switch (event) {
                case CONFIRM_ORDER -> SagaState.CONFIRMED;
                case CANCEL_REQUESTED -> SagaState.COMPENSATING;
                default -> invalid(state, event);
            };
            case COMPENSATING -> switch (event) {
                case COMPENSATION_COMPLETED -> SagaState.CANCELLED;
                case COMPENSATION_FAILED -> SagaState.FAILED_RECONCILIATION;
                default -> invalid(state, event);
            };
            case CONFIRMED, CANCELLED, FAILED_RECONCILIATION ->
                    invalid(state, event);
        };
    }

    public static boolean quorumSetsMustIntersect(int replicaCount,
            int firstAcknowledgements, int secondAcknowledgements) {
        if (replicaCount < 1
                || firstAcknowledgements < 0
```

JAVA (continued 9/12)

```
                || secondAcknowledgements < 0
                || firstAcknowledgements > replicaCount
                || secondAcknowledgements > replicaCount) {
            throw new IllegalArgumentException("invalid quorum sizes");
        }
        return firstAcknowledgements + secondAcknowledgements > replicaCount;
    }

    public static double errorBudgetEvents(
            long eligibleEvents, double sloTarget) {
        if (eligibleEvents < 0 || !Double.isFinite(sloTarget)
                || sloTarget < 0 || sloTarget > 1) {
            throw new IllegalArgumentException("invalid SLO inputs");
        }
        return eligibleEvents * (1.0 - sloTarget);
    }

    public static double burnRate(
            long badEvents, long eligibleEvents, double sloTarget) {
        if (badEvents < 0 || eligibleEvents <= 0
                || badEvents > eligibleEvents
                || !Double.isFinite(sloTarget)
                || sloTarget < 0 || sloTarget >= 1) {
            throw new IllegalArgumentException("invalid burn-rate inputs");
        }
        double observedBad = badEvents / (double) eligibleEvents;
        return observedBad / (1.0 - sloTarget);
    }

    public static void main(String[] args) {
        capacityAssertions();
        tokenBucketAssertions();
        retryAssertions();
```

JAVA (continued 10/12)

```
        offsetAssertions();
        sagaAssertions();
        quorumAndSloAssertions();
        System.out.println("DistributedSystemsPatterns assertions passed");
    }

    private static void capacityAssertions() {
        CapacityEstimate estimate = estimate(
                50_000_000, 8, 0.30, Duration.ofMillis(40),
                10_000_000, 1_536, 3);
        assert Math.abs(estimate.averageRps() - 578.7037) < 0.001;
        assert Math.abs(estimate.peakRps() - 4_629.6296) < 0.01;
        assert Math.abs(estimate.concurrentDependencyCalls() - 55.5555) < 0.01;
        assert estimate.replicatedTibPerYear() > 15;
    }

    private static void tokenBucketAssertions() {
        long start = 1_000_000_000L;
        var bucket = new TokenBucket(3, 2, start);
        assert bucket.tryAcquire(1, start);
        assert bucket.tryAcquire(2, start);
        assert !bucket.tryAcquire(1, start);
        long halfSecondLater = start + TimeUnit.MILLISECONDS.toNanos(500);
        assert bucket.tryAcquire(1, halfSecondLater);
        assert Math.abs(bucket.available(halfSecondLater)) < 1e-9;
        long farLater = start + TimeUnit.SECONDS.toNanos(10);
        assert Math.abs(bucket.available(farLater) - 3) < 1e-9;
    }

    private static void retryAssertions() {
        List<RetryAttempt> plan = retryPlan(
                4, Duration.ofMillis(500), Duration.ofMillis(80),
                Duration.ofMillis(20), Duration.ofMillis(100), 7);
```

JAVA (continued 11/12)

```
        assert !plan.isEmpty();
        assert plan.size() <= 3;
        Duration previousRemaining = Duration.ofMillis(500);
        for (RetryAttempt retry : plan) {
            assert retry.number() >= 2 && retry.number() <= 4;
            assert !retry.delay().isNegative();
            assert retry.remainingAfterDelay().compareTo(previousRemaining) < 0;
            previousRemaining = retry.remainingAfterDelay();
        }
    }

    private static void offsetAssertions() {
        var tracker = new ContiguousOffsetTracker(10);
        assert tracker.complete(11).isEmpty();
        assert tracker.nextCommitOffset() == 10;
        assert tracker.gapsSnapshot().equals(Set.of(11L));
        assert tracker.complete(10).orElseThrow() == 12;
        assert tracker.complete(10).isEmpty();
        assert tracker.complete(13).isEmpty();
        assert tracker.complete(12).orElseThrow() == 14;
    }

    private static void sagaAssertions() {
        SagaState state = SagaState.PENDING;
        state = transition(state, SagaEvent.INVENTORY_RESERVED);
        state = transition(state, SagaEvent.PAYMENT_TIMED_OUT);
        assert state == SagaState.PAYMENT_UNKNOWN;
        state = transition(state, SagaEvent.PAYMENT_CONFIRMED);
        state = transition(state, SagaEvent.CANCEL_REQUESTED);
        state = transition(state, SagaEvent.COMPENSATION_COMPLETED);
        assert state == SagaState.CANCELLED;

        try {
```

JAVA (continued 12/12)

```
            transition(SagaState.CONFIRMED, SagaEvent.CANCEL_REQUESTED);
            throw new AssertionError("terminal state accepted transition");
        } catch (IllegalStateException expected) {
            assert expected.getMessage().contains("CONFIRMED");
        }
    }

    private static void quorumAndSloAssertions() {
        assert quorumSetsMustIntersect(3, 2, 2);
        assert !quorumSetsMustIntersect(3, 1, 2);
        assert Math.abs(errorBudgetEvents(1_000_000, 0.999) - 1_000) < 1e-6;
        assert Math.abs(burnRate(1_000, 100_000, 0.999) - 10.0) < 1e-8;
    }

    private static SagaState invalid(SagaState state, SagaEvent event) {
        throw new IllegalStateException(
                "event " + event + " is invalid from " + state);
    }

    private static void requirePositiveFinite(double value, String name) {
        if (!Double.isFinite(value) || value <= 0) {
            throw new IllegalArgumentException(name + " must be positive");
        }
    }

    private static Duration requirePositive(Duration value, String name) {
        Objects.requireNonNull(value, name);
        if (value.isZero() || value.isNegative()) {
            throw new IllegalArgumentException(name + " must be positive");
        }
        return value;
    }
}
```

Part II - Practice and Handoff

Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: capacity, partition, replication, and message-log models
- Interview Core: consistency, Kafka, resilience, and observability
- SDE-2 Follow-up: hot keys, replay, compensation, and error budgets
- Challenge: lead a complete constraint-first system-design interview

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous volume: 18I - Advanced Java I: Persistence, SQL, JPA, and Caching

Series index

Return to the complete series index

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Complete the learning steps in order when rebuilding from fundamentals, or enter at the first step whose readiness checks expose a gap. Stable volume numbers remain in filenames; the steps below show the recommended reading order. Click a title when your PDF viewer permits local sibling-file links.

STEP	FOCUSED LEARNING PATH
1	Java Foundations for Problem Solving
2	Time and Space Complexity
3	Number Systems and Math Foundations for DSA Interviews
4	Bit Manipulation in Java
5	Loop Mastery, Patterns, and Index Calculations
6	Arrays and Array Problem-Solving Patterns
7	Strings and String Problem-Solving Patterns
8	Hashing: Maps, Sets, Frequency, and Prefix State
9	Recursion and Backtracking in Java
10	Linked Lists: Pointer Reasoning and Mutation
11	Stacks, Queues, Deques, and Monotonic Patterns
12	Binary Search: Bounds, Answers, and Invariants
13	Trees, Binary Search Trees, and Tries
14	Heaps, Priority Queues, Selection, and Top-K
15	Graphs: Traversal, Ordering, Paths, and Union-Find
16	Greedy Algorithms: Recognition, Proofs, and Scheduling
17	Dynamic Programming: State, Transitions, and Optimization
18	Advanced Java Engineering and the SDE-2 Interview (Parts A-J)

Learning Step 3 uses a linked Number Systems foundations volume and workbook; the final advanced stage uses a ten-part collection, including a three-volume backend specialist track. These splits keep every file focused and printable.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Volume 18 of 18 - Part J of J. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.